

Sophia Milask

Git Hub Essay for Probability and Applied Statistics

During the software development process, it is important to track changes in the source code. One way to track changes is by using Git, a distributed version control system which allows multiple users to work together on a project and maintain a history of all changes. There is no standardized way to use Git, instead, it offers the flexibility to manage changes by using workflows.

A Git workflow does not use a ridged set of instructions that each coder must follow but instead offers guidelines that the team will follow. These guidelines define how to interact with shared code, handle branches, resolve conflicts and deploy changes. The outcome of following these practices is said to streamline the development process, maintain quality and avoid errors.

There are multiple workflows that are available for teams to choose from and should reflect the teams' culture. The right workflow will enhance the team's effectiveness, but if it is not a good workflow, it can become a burden. Sometimes to get the best workflow for a team is by combining aspects of different workflows to match the values and goals of the team members, and making sure it does not add any unnecessary overhead.

The most common Git workflow is called the Centralized workflow. This is considered a great workflow for smaller teams or teams transitioning from another popular version control system called Subversion (SVN).

In a centralized workflow, usually there is a main branch. The Centralized workflow is created by making a central repository which represents the official project. This repository serves as the acknowledged source of truth for collaboration. is where the data is stored and contains the complete project history and all the branches.

Developers pull the source code from the from the main branch which makes a cloned copy of all files on their local system. They will make changes to the code on locally and

“push” those changes back to the repository. The push action copies the changed code to the repository where it can be reviewed before they are merged with the main branch. Before a developer can publish a feature, they need to pull down the latest version of the repository. The latest version of the repository is a “Commit”, which is a snapshot of the project files recoding all the code changes. Git will look at the local changes and see if they conflict with the variations that other team members have made.

If there are no conflicts, a “merge” command can be used to combine changes into the main branch. However, if there are conflicting changes, Git will pause the process and allow for manual resolution. A “merge conflict” occurs when more than one developer has made changes to the same file or set of files. Conflicts are caught each time a merge is made, rather than trying to address a single integration of several different

changes. This will keep the history of each commit focused and make it easier to find out if bugs were introduced.

Centralized workflows are not great for large teams, the more members involved, the greater the potential for conflicts. Therefore, it is worth exploring other workflows.

Feature Branching is an extension of the Centralized workflow. The idea is that all features should be in a dedicated branch and should not reside in the main branch. This makes it easier for multiple developers to work on a feature without interfering with the main branch. In this respect, the main branch will never contain broken code. A Gitflow Workflow is like feature branching, but assigns very specific roles to different branches, and defines how the branches should interact. A Forking Workflow is completely different whereas it does not use a central repository. Instead, each coder has two repositories, private local and public on a server.